

INSTITUTO FEDERAL DO AMAZONAS – CAMPUS MANAUS
ZONA LESTE

POLO DE INOVAÇÃO IFAM | DIGITAL TRANSFORMATION ACADEMY
DISCIPLINA: TÉCNICAS DE HYPERAUTOMATION

PDD – PROCESS DEFINITION DOCUMENT

**Documento de Definição de Processo: Fluxo de Desenvolvimento
Colaborativo com GitFlow no Portal Fake**

Aluno:	João Lucas Mota Ausier
Professor:	Moisés Levy
Semana / Módulo:	Semana 4 – Documentação de Engenharia e GitFlow (PDD)
Versão do Documento:	1.0
Nome do Processo:	Ciclo de Desenvolvimento Colaborativo, Versionamento e Integração Contínua com GitFlow

Data de Emissão:	28 de Agosto de 2026
-------------------------	----------------------

SUMÁRIO

1.

Objetivo do Processo
2. Escopo do Processo
3. Responsáveis e Papéis da Equipe
4. Descrição do Processo Atual (Sem Versionamento Estruturado)
5. Processo Futuro Proposto (Modelo GitFlow com Branches Padronizadas)
6. Volume e Demanda de Desenvolvimento
7. Frequência de Ciclos e Releases
8. Regras de Negócio e Políticas de Branching
9. Entradas do Processo
10. Saídas e Entregáveis do Processo
11. Mapeamento de Exceções, Conflitos de Merge e Erros Técnicos
12. Viabilidade Técnica e Justificativa de Engenharia
13. Retorno sobre o Investimento (ROI) e Benefícios Organizacionais
14. BPMN Associado ao Fluxo de Trabalho
15. Observações Finais e Recomendações Técnicas

1. OBJETIVO DO PROCESSO

Qual problema esse processo resolve? O processo soluciona os conflitos operacionais, sobreposição de código-fonte, instabilidades em produção e falta de rastreabilidade decorrentes de trabalhos simultâneos de múltiplos desenvolvedores nos

módulos de Cadastro, Consulta e Exportação do Portal Fake.

Qual o resultado esperado? Estabelecer um ciclo de desenvolvimento padronizado, seguro e auditável, garantindo que qualquer alteração de código passe por isolamento em branches dedicadas, commits semânticos, validações rigorosas em Pull Requests e homologação antes da entrega final.

Qual o objetivo final? Implementar a estratégia de versionamento **GitFlow** suportada pela plataforma GitHub, assegurando a integridade contínua da branch estável (main), a integração estruturada na branch de homologação (develop) e a entrega síncrona dos módulos automatizados da aplicação.

2. ESCOPO DO PROCESSO

O escopo estabelece as fronteiras operacionais e técnicas do ciclo de engenharia de software da equipe:

- **Início (Gatilho):** Criação do repositório remoto no GitHub pelo Líder Técnico, configuração da branch develop e delegação das tarefas de desenvolvimento de módulos aos integrantes da equipe.
- **Fim:** Validação de integração pelo Líder Técnico, aprovação de todos os Pull Requests, realização do Merge da develop para a main e entrega oficial da versão consolidada (Sistema Entregue).

3. RESPONSÁVEIS E PAPÉIS DA EQUIPE

Papel / Agente	Tipo	Responsabilidades Técnicas
Líder Técnico (Tech Lead)	Humano / Gestão	Criar repositório, configurar branches de infraestrutura (main e develop), revisar Pull

Papel / Agente	Tipo	Responsabilidades Técnicas
		Requests (Code Review), validar testes de integração, gerenciar conflitos e autorizar merges finais.
Desenvolvedor de Cadastro	Humano / Engenharia	Clonar repositório, criar a branch feature/cadastro, codificar scripts de automação/modal de cadastro, efetuar commits, enviar push e abrir Pull Request para develop.
Desenvolvedor de Consulta	Humano / Engenharia	Clonar repositório, criar a branch feature/consulta, codificar as rotinas de busca rápida, filtros por status, CPF e e-mail, versionar e abrir Pull Request para develop.
Desenvolvedor de Exportação	Humano / Engenharia	Clonar repositório, criar a branch feature/exportacao, implementar fluxos de extração de dados (CSV e JSON) e rotinas de carga massiva, versionar e submeter Pull Request para develop.

4. DESCRIÇÃO DO PROCESSO ATUAL (SEM VERSIONAMENTO)

ESTRUTURADO)

No modelo manual e não padronizado, os desenvolvedores trabalhavam diretamente em arquivos locais isolados ou submetiam commits sem revisão diretamente na branch principal (main). Esse fluxo apresentava os seguintes problemas:

1. Código instável enviado diretamente para o ambiente principal, quebrando funcionalidades já entregues.
2. Sobrescrita acidental de código-fonte entre integrantes que trabalhavam no mesmo arquivo HTML/JavaScript do Portal Fake.
3. Falta de rastreabilidade sobre qual membro da equipe introduziu determinada falha ou alteração.
4. Dificuldade de integrar os módulos simultâneos de Cadastro, Consulta e Exportação sem paralisar o trabalho da equipe.

5. PROCESSO FUTURO PROPOSTO (MODELO GITFLOW COM BRANCHES PADRONIZADAS)

O novo processo estrutura o fluxo colaborativo seguindo rigorosamente a arquitetura **GitFlow**:

1. **Setup e Configuração Inicial:** O Líder Técnico cria o repositório central no GitHub, gera a branch de integração develop a partir da main, adiciona os desenvolvedores como colaboradores e compartilha o link do repositório.
2. **Abertura em Paralelo (Fork):** Por meio de um Gateway Paralelo, os desenvolvedores iniciam suas atividades de forma concorrente e isolada:
 - Clonam o repositório remoto em seus ambientes locais de desenvolvimento.
 - Criam suas respectivas branches temáticas: feature/cadastro, feature/consulta e feature/exportacao a partir de develop.

- Desenvolvem e testam as automações e funcionalidades designadas.
 - Registram o progresso por meio de commits atômicos estruturados e realizam git push para o GitHub.
 - Abrem um **Pull Request (PR)** apontando para a branch develop.
3. **Sincronização e Code Review:** O Líder Técnico recebe a notificação dos PRs, analisa a conformidade do código, executa os testes de integração entre os módulos e realiza o Merge para a branch develop.
4. **Gateway de Aprovação (XOR):**
- **Se Não Aprovado:** O Líder solicita correções diretamente no Pull Request, devolvendo as branches aos desenvolvedores para ajustes até a conformidade total.
 - **Se Aprovado:** O Líder realiza o Merge da branch develop para a branch main, taggeia a versão e declara o sistema formalmente entregue.

6. VOLUME E DEMANDA DE DESENVOLVIMENTO

- ---

Volumetria de Branches: Pelo menos 3 branches de funcionalidade ativas em paralelo (feature/*), além das branches permanentes main e develop.
- **Frequência de Commits e PRs:** Dezenas de commits diários durante o ciclo de desenvolvimento (sprints) e pelo menos 3 a 5 Pull Requests completos por ciclo de entrega de versão.

7. FREQUÊNCIA DE CICLOS E RELEASES

Ciclos Semanais / Contínuos. A criação e manutenção de branches de desenvolvimento ocorrem diariamente, com consolidação de branches de integração

(develop) e fechamento de releases na branch main a cada encerramento de sprint ou entrega de módulo da disciplina.

8. REGRAS DE NEGÓCIO E POLÍTICAS DE BRANCHING

Regra	Descrição e Validação Técnica
RN01 – Proteção da Branch Main	É estritamente proibido realizar commits ou <i>pushes</i> diretos na branch main. Ela deve conter apenas código estável, testado e originado exclusivamente por merge da develop.
RN02 – Isolamento de Features	Todo novo desenvolvimento deve ser criado em uma branch com prefixo padronizado (ex: feature/cadastro) derivada a partir da develop mais recente.
RN03 – Obrigatoriedade de Pull Request	Nenhuma branch feature/* pode ser incorporada à develop sem a abertura de um Pull Request com descrição detalhada das modificações.
RN04 – Revisão por Par Obrigatória (Code Review)	O Líder Técnico deve revisar e aprovar o PR antes de qualquer merge, verificando conformidade com as boas práticas e ausência de bugs.
RN05 – Mensagens de Commit Semânticas	Os commits devem ser atômicos e seguir mensagens claras indicando a intenção da alteração (ex: " <i>feat(cadastro): implementa automacao com selenium</i> ").

9. ENTRADAS DO PROCESSO

Entrada	Tipo	Origem / Ferramenta	Descrição
Repositório Remoto	Serviço Git / Cloud	GitHub	Espaço central contendo a branch main e develop inicializadas.
Código-fonte das Automações	Scripts Python / HTML / JS	Ambiente Local (IDE)	Scripts Selenium e modificações do Portal Fake desenvolvidos por cada integrante.
Pull Requests (PRs)	Requisição de Merge	GitHub Interface	Solicitação de incorporação das branches feature/* para a develop.

10. SAÍDAS E ENTREGÁVEIS DO PROCESSO

- ---

Repositório Sincronizado e Integrado: Repositório GitHub contendo todo o histórico de versionamento com branches organizadas.
- **Branch Main Estável:** Código-fonte consolidado, testado e pronto para uso em produção contendo os três módulos integrados.
- **Documentação e Rastreabilidade:** Histórico completo de commits, discussões de Pull Requests e revisões de código aprovadas.
- **Arquivos de Modelagem e PDD:** Diagramas BPMN (.drawio e .png) e especificações técnicas arquivados nas pastas correspondentes do repositório.

11. MAPEAMENTO DE EXCEÇÕES, CONFLITOS DE MERGE E ERROS TÉCNICOS

Exceção / Erro	Causa Raiz	Tratamento Operacional e Técnico
Merge Conflict (Conflito de Merge)	Alterações concorrentes nas mesmas linhas de código por desenvolvedores diferentes.	O Líder Técnico solicita ao desenvolvedor a sincronização da sua branch local com a develop mais recente (git pull origin develop) para resolução manual dos marcadores de conflito antes do merge.
Reprovação no Code Review	Inconformidade técnica, erros de sintaxe ou falta de cobertura de requisitos no módulo.	O Líder Técnico rejeita o PR via Gateway Exclusivo (caminho 'Não'), aponta comentários na linha de código e o desenvolvedor realiza novos commits corretivos na mesma branch de feature.
Quebra na Integração	Incompatibilidade entre a automação de cadastro e os filtros de consulta/exportação.	Execução de testes conjuntos em ambiente de staging/local; reversão do commit instável caso necessário antes de qualquer push para a branch main.

12. VIABILIDADE TÉCNICA E JUSTIFICATIVA DE ENGENHARIA

A adoção do GitFlow é viável e recomendada? [X] SIM [] NÃO

Justificativa Técnica: O GitFlow é o padrão amplamente aceito na indústria de software

para gerenciar projetos colaborativos concorrentes. A sua utilização no desenvolvimento da automação do Portal Fake reduz a zero o risco de sobrescrita de código, formaliza a etapa de testes pré-integração e garante que apenas artefatos completamente validados alcancem a branch main, assegurando alto nível de maturidade de engenharia de software e Hyperautomation.

13. RETORNO SOBRE O INVESTIMENTO (ROI) E BENEFÍCIOS ORGANIZACIONAIS

A implementação do fluxo GitFlow proporciona ganhos estratégicos expressivos para a equipe técnica:

- **Paralelismo Real de Desenvolvimento:** Permite que três ou mais desenvolvedores atuem simultaneamente nos módulos de Cadastro, Consulta e Exportação sem bloqueios operacionais.
- **Redução de Retrabalho por Conflitos:** Mitigação drástica de tempo gasto recuperando código sobrescrito manualmente.
- **Qualidade Assegurada via Code Review:** Aumento da qualidade e robustez do código devido à checagem prévia em todos os Pull Requests pelo Líder Técnico.
- **Rastreabilidade e Compliance:** Capacidade de auditar o autor, a data e a motivação exata de cada linha modificada no sistema.

14. BPMN ASSOCIADO AO FLUXO DE TRABALHO

O fluxo operacional e colaborativo está integralmente modelado no arquivo r04_portal_fake_gitflow_bpmn.drawio, estruturado com os seguintes componentes:

- **1 Pool Principal:** Portal Fake - Processo de Desenvolvimento Colaborativo com GitFlow.

- **4 Lanes Especializadas:** Líder Técnico, Desenvolvedor de Cadastro, Desenvolvedor de Consulta e Desenvolvedor de Exportação.
- **Gateways Paralelos (AND):** Sincronização da abertura e do fechamento dos desenvolvimentos em paralelo.
- **Gateway Exclusivo (XOR):** Validação de aprovação do sistema e desvio para ciclo corretivo.

[Área Reservada para Inserção da Imagem do Diagrama BPMN:

r04_portal_fake_gitflow_bpmn.png]

15. OBSERVAÇÕES FINAIS E RECOMENDAÇÕES TÉCNICAS

1.

Automação com CI/CD: Recomenda-se integrar o repositório ao GitHub Actions para executar testes unitários e linters automatizados em cada abertura de Pull Request.
 2. **Branches de Release e Hotfix:** Em cenários de grande escala, adicionar as ramificações de release/* e hotfix/* previstas na especificação estendida do GitFlow.
 3. **Organização de Repositório:** Manter a estrutura de pastas padronizada (ex.: Semana01_Hyperautomation, Semana02_Hyperautomation, Semana03_Hyperautomation e Semana04_PDD) para garantir a consistência das entregas acadêmicas e profissionais.
-

Documento PDD – GitFlow Portal Fake | Elaborado por João Lucas Mota Ausier | Disciplina:
Técnicas de Hyperautomation